



دانشگاه شاهد

Shahed University

آنالیز شبکه های اجتماعی

Social Network Analysis

استاد راهنما :

خانم دکتر پوربهمن

توسعه دهندگان :

امیدرئیزی

امیرمهدی زارعان

امیرحسین شورابی

علی فلاح

در دنیای امروز، شبکه‌های اجتماعی نقش بسیار مهمی در ارتباطات ایفا می‌کنند و بررسی ساختار این ارتباطات برای سازمان‌ها و شرکت‌ها ارزش بالایی دارد. هدف از اجرای این پروژه، طراحی و توسعه یک ابزار نرم‌افزاری مستقل برای استخراج، تحلیل و پردازش اطلاعات شبکه‌های اجتماعی است. در این سیستم، شبکه اجتماعی به شکل یک تئوری گراف مدل‌سازی شده است؛ به این صورت که کاربران به عنوان راس‌های گراف و روابط دوستی میان آن‌ها به عنوان یال‌های گراف در نظر گرفته شده‌اند.

هسته اصلی این نرم‌افزار وظیفه دارد پردازش‌های زیر را با کمترین پیچیدگی زمانی ممکن انجام دهد

نمایش دوستان : لیست کردن تمام دوستان یک کاربر مشخص .

بررسی ارتباط : تشخیص اینکه آیا دو کاربر در شبکه با یکدیگر ارتباط (دوستی) دارند یا خیر .

مسیریابی : پیدا کردن کوتاه‌ترین مسیر ممکن بین دو کاربر .

سیستم پیشنهاد دوست : ارائه پیشنهاد‌های دوستی به کاربران بر اساس ساختار گراف .

تشخیص گروه‌ها : شناسایی و لیست کردن گروه‌های مختلف شبکه (مولفه‌های همبند) و اعضای آن‌ها .

کاربران محبوب : یافتن کاربری که بیشترین تعداد دوست را در سطح شبکه دارد .

دوستان مشترک : پیدا کردن دوستان مشترک میان دو کاربر .

آمار و اطلاعات کلان شبکه : محاسبه تعداد کل کاربران، تعداد کل دوستی‌ها، میانگین روابط به ازای هر کاربر، یافتن بزرگترین گروه دوستی ، شناسایی کاربر با بیشترین دوست ، قطر گراف و چگالی گراف .

محاسبه فواصل : محاسبه فاصله یک کاربر خاص از تمامی افراد شبکه و مرتب‌سازی آن‌ها از کمترین تا بیشترین فاصله .

یافتن افراد کلیدی (پل‌های ارتباطی) : شناسایی کاربرانی که بیشترین مرکزیت بینایی را دارند؛ به این معنا که بیشترین حضور را در کوتاه‌ترین مسیرهای شبکه دارند و حذف آن‌ها باعث از هم پاشیدن شبکه می‌شود.

تشخیص جوامع دوستی پیشرفته : توانایی تفکیک اکیپ‌ها و جوامع فشرده در دل یک گروه بزرگتر (زیرگراف‌هایی با تراکم یال بالا درونی و یال‌های خروجی اندک).

بیشینه‌سازی انتشار خبر : شناسایی کاربر بهینه در شبکه که در صورت دریافت یک خبر، می‌تواند آن را در کمترین زمان ممکن به بیشترین تعداد افراد شبکه برساند .

نکات کلی درباره توسعه این سیستم

ذخیره‌سازی داده‌ها : اطلاعات مربوط به شبکه اجتماعی در قالب فایل‌های JSON ذخیره و بازیابی می‌شوند.

مدیریت داده‌ها : سیستم قابلیت افزودن، ویرایش و حذف کاربران را به صورت بهینه دارا می‌باشد.

اولویت توسعه : هدف اصلی در تمام بخش‌های سیستم، استفاده از داده‌ساختارها و الگوریتم‌های بهینه برای مینیمم کردن پیچیدگی زمانی پردازش‌ها بوده است.

معماری نرم‌افزار

در پروژه‌های تحلیل شبکه، الگوریتم‌های پردازش گراف بسیار سنگین و پیچیده هستند. اگر منطق این الگوریتم‌ها با فرمت‌های نمایش در رابط کاربری (UI) یا منطق تبدیل داده‌ها (مانند تبدیل آیدی به نام کاربر) ترکیب شود، کد به شدت شکننده شده و با کوچک‌ترین تغییر در UI یا نحوه دریافت ورودی، الگوریتم‌های گراف دچار خطا می‌شوند.

به همین دلیل، در این پروژه از الگوی **معماری تمیز (clean Architecture)** استفاده شده است. فلسفه اصلی این معماری جداسازی کامل دغدغه‌ها است.

1- لایه Core (هسته پردازشی و الگوریتم‌ها) :

این لایه از 4 بخش مختلف تشکیل شده است ، در بخش اول یعنی Model گراف ما ذخیره شده است برای دسترسی راحت تر و اجرای یک سری متد های خاص . در بخش دوم یعنی Results خروجی تمامی الگوریتم ها در قالب یک کلاس طراحی شده است برای توسعه بهتر . در بخش سوم این لایه که Algorithms هست پیاده سازی تمامی الگوریتم های مورد نیاز در پروژه انجام شده که اینترفیس های این الگوریتم ها در بخش چهارم یعنی Abstractions قرار دارد.

2 - لایه Application :

در بخش اول این لایه (services) تمامی سرویس های مورد نیاز برای پروژه به دو قسمت دسته بندی شده اند ، یکی Queries که مربوط به سرویس هایی هستند که فقط اطلاعات را از گراف میخوانند و هیچ تغییری در گراف نمیدهند و یکی Commands که مربوط به سرویس هایی هست که گراف را تغییر میدهند مثل اضافه کردن کاربر یا دوستی . بخش دوم DTO ها هستند که کلاس هایی هستند که کنترل میکنند خروجی هر الگوریتم متناسب با نیاز لایه UI به دست این لایه برسد . بخش سوم Abstractions هست که اینترفیس تمامی سرویس های ما در این بخش قرار دارد . بخش چهارم یعنی contracts اینترفیس کلاس هایی هستند که پیاده سازی آن در لایه دیگری انجام شده اما در این لایه استفاده میشود.

3 - لایه (UI) Presentation :

این لایه فقط و فقط وظیفه گرفتن ورودی از کاربر و رندر کردن DTO های دریافتی از لایه Application را دارد. هیچ گونه الگوریتم گراف یا محاسبه ریاضی در این لایه وجود ندارد.

4 - لایه Infrastructure :

بخش اول این لایه به نام Storage نحوه کار با فایل های json را مشخص میکند . بخش دوم یعنی Persistence اطلاعاتی که از فایل های json گرفته شده را مدیریت میکند و بخش سوم یعنی Runtime قوانین مربوط به Persistence را تعیین میکند و اطلاعات را در RAM نگهداری میکند .

تشریح الگوریتم‌ها و تحلیل پیچیدگی زمانی و فضایی

در این فصل، هسته مرکزی پردازش‌های سیستم و منطق ریاضی پشت هر یک از نیازمندی‌های شبکه اجتماعی مورد بررسی قرار می‌گیرد. با توجه به اهمیت سرعت اجرا و مصرف بهینه منابع حافظه در گراف‌های مقیاس بزرگ، تمامی عملیات‌ها از پایه و بدون وابستگی به کتابخانه‌های آماده طراحی شده‌اند. در ادامه، روند کار دقیق و تحلیل نماد O بزرگ (Big-O) برای هر یک از الگوریتم‌های پیاده‌سازی شده در سیستم به تفکیک ارائه و اثبات می‌شود.

1 - BFS :

هدف این متد، شروع حرکت از یک گره مبدا و پیمایش گراف به صورت لایه به لایه است. همزمان با این پیمایش، فاصله دقیق (تعداد یال‌ها) از گره مبدا تا تمام گره‌های متصل به آن نیز محاسبه و ذخیره می‌شود.

1. مقداردهی اولیه : ابتدا سه ساختمان داده کلیدی ایجاد می‌شوند:

○ یک Queue (صف) برای مدیریت گره‌هایی که باید پردازش شوند.

○ یک HashSet برای ثبت گره‌های ملاقات شده (جلوگیری از افتادن در حلقه بی‌نهایت).

○ یک Dictionary برای نگهداری فاصله هر گره تا مبدا.

2. گره مبدا وارد صف و مجموعه visitedSet شده و فاصله آن تا خودش برابر 0 تنظیم می‌شود.

3. حلقه اصلی پیمایش : تا زمانی که صف خالی نشده است، گره جلوی صف خارج می‌شود.

4. لیست دوستان (همسایگان) این گره از ساختار گراف دریافت می‌شود.

5. به ازای هر دوست، اگر قبلاً ملاقات نشده باشد، به عنوان یک گره جدید علامت‌گذاری شده، وارد صف می‌شود و فاصله آن دقیقاً برابر با فاصله گره فعلی به علاوه یک ثبت می‌گردد.

6. در نهایت، سیستم شیء BFSResult را که حاوی لیست گره‌های پیمایش شده و دیکشنری فواصل آن‌هاست، برمی‌گرداند.

پیچیدگی زمان (Time Complexity) :

در این الگوریتم، هر گره (کاربر) دقیقاً یک بار وارد صف و از آن خارج می‌شود. همچنین در طول اجرای کامل الگوریتم، هر یال (رابطه دوستی) متعلق به گره‌های متصل به مبدأ، حداکثر دو بار بررسی می‌شود (چون گراف بدون جهت است). از آنجایی که عملیات بررسی وجود یک گره در HashSet و افزودن به آن دارای زمان اجرای ثابت $O(1)$ است، پیچیدگی زمانی نهایی الگوریتم برابر با $O(V + E)$ خواهد بود که در آن V تعداد کاربران (گره‌ها) و E تعداد روابط دوستی (یال‌ها) در مؤلفه همبند مربوطه است. این بهینه‌ترین زمان ممکن برای پیمایش یک گراف بدون وزن است.

پیچیدگی فضایی :

الگوریتم BFS از یک Queue برای نگهداری گره‌های در صف، یک HashSet برای ثبت گره‌های بازدید شده و معمولاً یک Dictionary برای ذخیره فاصله یا والد هر گره استفاده می‌کند. در بدترین حالت، همه گره‌ها در این ساختارها ذخیره می‌شوند؛ بنابراین پیچیدگی فضایی $O(V)$ است.

2 - DFS :

1. مقداردهی اولیه : ابتدا یک Stack (پشته) برای مدیریت گره‌ها با منطق LIFO (آخرین ورودی، اولین خروجی)، یک HashSet برای پیگیری گره‌های ملاقات شده، و یک List برای ذخیره ترتیب گره‌های پیمایش شده ایجاد می‌شود.
2. گره مبدأ وارد HashSet (علامت گذاری به عنوان ملاقات شده) و پشته می‌شود.
3. حلقه اصلی پیمایش : تا زمانی که پشته خالی نشده است، بالاترین گره از پشته خارج (Pop) شده و به لیست نتیجه اضافه می‌شود.
4. لیست دوستان (همسایگان) این گره از گراف دریافت می‌شود.
5. به ازای تک تک دوستان، اگر گره مورد نظر در visitedSet وجود نداشته باشد، بلافاصله به عنوان ملاقات شده علامت می‌خورد و وارد پشته (Push) می‌شود. به دلیل ماهیت LIFO در پشته، آخرین همسایه‌ای که وارد پشته می‌شود، در دور بعدی حلقه بلافاصله پردازش شده و حرکت در عمق گراف ادامه می‌یابد.

پیچیدگی زمانی (Time Complexity) :

استراتژی ثبت گره‌ها در visitedSet دقیقاً پیش از ورود به پشته (Push)، تضمین می‌کند که هیچ گرهی بیش از یک بار وارد پشته نشود. در نتیجه، حلقه بیرونی (while) دقیقاً به تعداد گره‌های قابل دسترس (V) تکرار می‌شود. در حلقه داخلی، تمام یال‌های متصل به گره فعلی بررسی می‌شوند. از آنجایی که در یک گراف بدون جهت هر یال دقیقاً دو بار (یک بار از سمت هر یک از دو گره متصل به آن) بررسی می‌شود، مجموع بررسی یال‌ها برابر با $2E$ خواهد بود. با توجه به اینکه تمام عملیات‌های درج در HashSet، جستجو در آن و کار با پشته دارای مرتبه زمانی ثابت $O(1)$ هستند، پیچیدگی زمانی کل این الگوریتم به بهینه‌ترین حالت ممکن یعنی $O(V + E)$ می‌رسد.

پیچیدگی فضایی :

DFS از یک Stack یا پشته بازگشتی برای مدیریت پیمایش و یک HashSet برای نگهداری گره‌های بازدید شده استفاده می‌کند. همچنین ممکن است ترتیب پیمایش در یک List ذخیره شود. در بدترین حالت، حافظه مورد نیاز متناسب با تعداد گره‌ها خواهد بود؛ بنابراین پیچیدگی فضایی $O(V)$ است.

3 - Adamic Adar :

این الگوریتم یکی از معیارهای هوشمندانه و قدرتمند در تئوری گراف برای پیشنهاد دوست و سنجش میزان ارتباط دو گره است. ایده اصلی این روش بر این منطق استوار است که دوستان مشترکی که خلوت‌تر هستند و دوستان کمتری دارند، نشان‌دهنده پیوند عمیق‌تری بین دو فرد هستند. تشریح روند کار :

این الگوریتم میزان شباهت بین دو گره ورودی nodeA و nodeB را طبق مراحل زیر محاسبه می‌کند:

1. اعتبارسنجی ورودی‌ها: (Validation)

- اگر هر دو گره یکسان باشند ($nodeA == nodeB$)، امتیاز صفر برگردانده می‌شود.
- وجود هر دو گره در شبکه و داشتن حداقل یک دوست برای هر کدام بررسی می‌شود. در صورت عدم برقرار بودن این شرایط، امتیاز صفر بازگردانده می‌شود.

2. استخراج دوستان مشترک: (Finding Common Friends)

- لیست دوستان nodeA و nodeB استخراج می‌شود.
- برای بهینه‌سازی سرعت جستجو، دوستان nodeB وارد یک HashSet با زمان دسترسی $O(1)$ می‌شوند.
- با پیمایش روی دوستان nodeA و شرط بررسی وجود در HashSet مربوط به nodeB، تمامی دوستان مشترک در یک لیست ذخیره می‌شوند.

3. محاسبه امتیاز وزن‌دار: (Scoring)

- به ازای هر دوست مشترک، درجه (تعداد کل دوستان) آن گره محاسبه می‌شود.
- اگر درجه آن دوست مشترک بزرگتر از ۱ باشد، وزن معکوس لگاریتمی آن با فرمول $\frac{1}{\log(\text{degree})}$ محاسبه شده و به totalScore اضافه می‌شود.

4. بازگرداندن نتیجه: مجموع امتیازات محاسبه‌شده به عنوان وزن شباهت دو گره برگردانده می‌شود.

تحلیل پیچیدگی زمانی (Time Complexity):

فرض کنیم d_A درجه (تعداد دوستان) گره A و d_B درجه گره B باشد:

- دریافت لیست‌ها و تبدیل به HashSet: دریافت لیست همسایگان در ساختار لیست مجاورت زمان $O(1)$ دارد. تبدیل لیست دوستان گره B به HashSet دقیقاً نیازمند $O(d_B)$ عملیات است.
- پیدا کردن دوستان مشترک: پیمایش روی d_A دوست گره A و چک کردن هر کدام در HashSet (با زمان ثابت $O(1)$)، زمانی برابر با $O(d_A)$ می‌برد.
- محاسبه امتیاز لگاریتمی: تعداد دوستان مشترک حداکثر برابر با $\min(d_A, d_B)$ است. به ازای هر دوست مشترک، تابع $\text{Count}()$ زمان $O(1)$ برمی‌گرداند و محاسبه لگاریتم نیز زمان ثابت $O(1)$ دارد.

بنابراین، پیچیدگی زمانی کل الگوریتم برابر با $O(d_A + d_B)$ است. این بدین معناست که اجرای الگوریتم کاملاً مستقل از کل اندازه شبکه بوده و فقط و فقط به تعداد دوستان دو گره مورد نظر وابسته است که باعث سرعت اجرای بالای آن در مقیاس‌های بزرگ می‌شود.

پیچیدگی فضایی:

برای محاسبه Adamic-Adar، همسایگان دو گره هدف در ساختارهایی مانند HashSet یا List ذخیره می‌شوند تا دوستان مشترک شناسایی شوند. بنابراین فضای کمکی مورد نیاز به درجه دو گره وابسته است :

$$O(\max(\deg(A), \deg(B)))$$

در صورتی که نتایج برای همه زوج‌گره‌ها ذخیره شوند، فضای خروجی می‌تواند تا $O(V^2)$ افزایش پیدا کند.

4 - Average Degree :

روند کار : این الگوریتم ابتدا تمام کاربران شبکه را پیمایش کرده، تعداد دوستان (درجه) هر کاربر را حساب و با بقیه جمع می‌کند. در نهایت حاصل جمع کل دوستی‌ها بر تعداد کاربران شبکه تقسیم می‌شود تا میانگین روابط به ازای هر فرد مشخص گردد.

تحلیل پیچیدگی زمانی (Time Complexity) :

پیمایش تمامی گره‌های شبکه به $O(V)$ زمان نیاز دارد و گرفتن تعداد دوستان هر گره در زمان ثابت $O(1)$ انجام می‌شود. بنابراین پیچیدگی زمانی کل الگوریتم برابر با $O(V)$ است .

پیچیدگی فضایی :

میانگین درجه از مجموع درجه گره‌ها و تعداد گره‌ها محاسبه می‌شود. اگر فقط مقدار نهایی میانگین ذخیره شود، چند متغیر عددی مانند sum و count کافی است؛ بنابراین فضای کمکی $O(1)$ است.

اگر درجه تمام گره‌ها ابتدا در یک آرایه یا Dictionary ذخیره شود، فضای مورد نیاز $O(V)$ خواهد بود.

: Average Path Length - 5

این الگوریتم میانگین طول کوتاه‌ترین مسیر بین تمامی جفت کاربران دارای ارتباط در شبکه را محاسبه می‌کند:

1. اعتبارسنجی اولیه: اگر تعداد کل گره‌های شبکه ۱ یا کمتر باشد، میانگین مسیر صفر برگردانده می‌شود.
2. اجرای BFS برای تمامی گره‌ها: به ازای هر کاربر در شبکه (startNode)، الگوریتم پیمایش سطح اول (BFS) اجرا می‌شود تا فاصله کوتاه‌ترین مسیر از آن کاربر به تمامی گره‌های قابل دسترس مشخص گردد.
3. محاسبه مجموع فواصل: فواصل تمام گره‌های مقصد (به جز خود گره مبدا) در متغیر sumOfAllPaths جمع شده و تعداد جفت‌های مرتبط در reachablePairsCount شمارش می‌شود.
4. محاسبه میانگین: در نهایت با تقسیم مجموع فواصل بر تعداد کل جفت‌های قابل دسترس، میانگین طول مسیر کل شبکه برگردانده می‌شود (در صورت عدم وجود هرگونه مسیر، مقدار صفر برمی‌گردد).

تحلیل پیچیدگی زمانی (Time Complexity):

- اجرای الگوریتم BFS برای یک گره به تنهایی نیازمند $O(V + E)$ زمان است.
- چون الگوریتم BFS به ازای تک‌تک گره‌های شبکه اجرا می‌شود و حلقه جمع‌آوری فواصل نیز به ازای هر گره زمانی حداکثر $O(V)$ می‌برد، پیچیدگی زمانی کل الگوریتم برابر با $O(V \times (V + E))$ یا به عبارتی $O(V^2 + VE)$ خواهد بود.

پیچیدگی فضایی:

برای محاسبه میانگین طول مسیر، معمولاً از BFS یا DFS از هر گره استفاده می‌شود. در هر اجرای پیمایش، ساختارهایی مانند Queue، HashSet و Dictionary برای ذخیره گره‌های بازدید شده و فاصله‌ها استفاده می‌شوند. بنابراین اگر پیمایش‌ها به صورت جداگانه انجام شوند، فضای کمکی $O(V)$ است.

اگر تمام فاصله‌های بین همه زوج گره‌ها ذخیره شوند، پیچیدگی فضایی به شکل زیر خواهد بود:

$$O(V^2)$$